Exploring Twitter System Design - Part 5

DAY 109
27/10/25

Home Timeline (Read path & Fan-out strategy)

a. Approaches

- **Fan-out on write (push)**: On `TweetCreated`, insert into each follower's TimelineEntry list.
  Pros: fast reads; Cons: heavy writes; hot spots on celebs.
- **Fan-out on read (pull)**: Read follows, fetch latest N from each, merge-sort + rank.
  Pros: fewer writes; Cons: slow reads, heavy fan-in.
- **Hybrid**: Push for users below threshold (e.g., < 100k followers); for celebrities, compute on read and cache.

b. Read Flow

1. Client calls GET /timeline/home?cursor=....
2. Check cache (Redis): materialized page for user. On miss, fetch TimelineEntries (pinned lists) and rank.
3. Join with tweet bodies (batch `mget`), filter blocks/mutes, apply visibility.
4. Return paginated results + `nextCursor` (opaque).

c. Ranking (MVP → ML)

- **MVP heuristic**: recency + engagement (likes/retweets/replies) + affinity(you↔author) + diversity (no floods).
- **Online features**: freshness minutes, author interactions, user activity, text/hashtag signals.
- **ML path**: train on clicks/likes dwell-time; online feature store fed by streams; A/B experiments with guardrails.

d. Caching

- Per-user home timeline pages (e.g., 100–200 entries/page) TTL ~ minutes; invalidation on new tweets from follows.
- Hot tweet bodies cached by `tweetId` with short TTL + write-through on update.

Search & Trends

a. Indexing

- Stream processor tokenizes text/entities; build inverted index (term → postings list of tweetIds with positions).
- Tiered index: realtime (seconds) + archive segments; rolling merges.
- Geo/language fields for filtering.

b. Query

- GET /search?q=...&type=tweets|users&sort=top|latest&lang=...&geo=...
- Parse → rewrite (spelling, synonyms, hashtags) → candidate fetch → rank (BM25/ML) →

safety filters (blocks/mutes/NSFW).

c. Trends

- Rolling counters per region/language; exponential decay window; exclude spam/bot patterns; output top-K per bucket every few minutes.

6.17 Engagements, Replies, Threads

- Likes/retweets recorded as append-only events; maintain denormalized counters with CRDTs or transactional increments per shard + periodic reconcile.
- Replies form a conversation graph; store `inReplyToId`, compute thread views lazily with capped depth and pagination.

6.18 Social Graph

- Follows are directed edges; partitions by followerId ("who I follow") and by followeeId ("my followers") with secondary views for fan-out.
- Suggestion: maintain mutuals cache for UI and abuse heuristics.

6.19 Media Pipeline

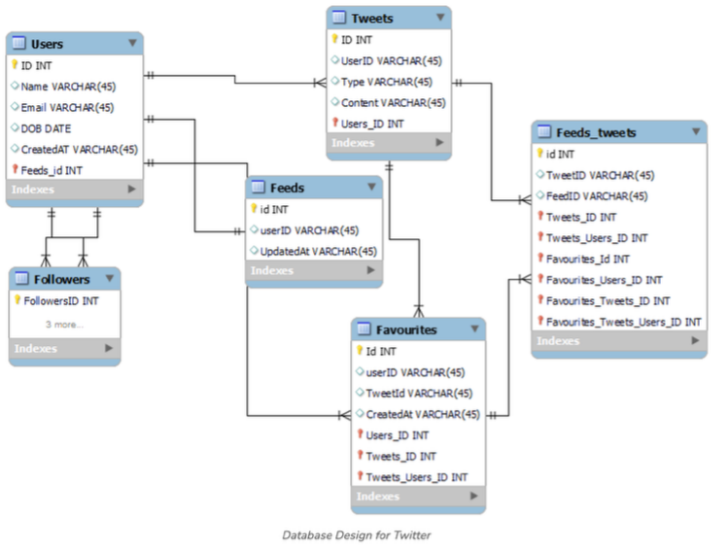
- Upload via signed URL to object store; virus scan; transcode variants (image sizes, video bitrates); write metadata rows.
- Serve via CDN with cache-control; origin shield to protect storage; hash-based keys for dedupe.

6.20 Notifications

- Subscribe to events (`LikeCreated`, `FollowCreated`, `Mentioned`, `ReplyCreated`).
- Fanout through notification preference rules; enqueue to APNs/FCM/Email/SMS.
- Per-user inbox store for in-app notifications; read markers and collapse logic.

7. Data Model Design for Twitter System Design

This is the general Dara model which reflects our requirements.



Database Design for Twitter

The diagram have the following table

Users
Tweets
Feeds
Followers
Feeds_tweets
Favourites

7.1 Users:

In this table contain a user's information such name, email, DOB, and other details where ID will be autofield and it will be unique.

```
Users
{
ID: Autofield,
Name: Varchar,
Email: Varchar,
DOB: Date,
Created At: Date
}
```

- Fields: `userId` (Snowflake int64), `handle` (unique), `name`, `email` (unique), `bio`, `avatarUrl`, `bannerUrl`, `createdAt`, `updatedAt`, `settings` (JSON: language, privacy, dmPrefs…), `statusFlags` (bitmask: suspended, protected, deactivated, suspectedBot), counters `{followersCount, followingCount, tweetsCount, likesCount}` (denormalized, eventually consistent).
- Typical queries: by `userId` (profile fetch), by `handle` (login/lookup).
- Consistency: strong for `handle`, `email`; eventual for counters.

7.2 Tweets:

As the name suggests, this table will store tweets and their properties such as type (text, image, video, etc.) content etc. UserID will also store.

```
Tweets
{
id: uuid,
UserID: uuid,
type: enum,
content: varchar,
createdAt: timestamp
}
```

- Fields: `tweetId` (Snowflake), `authorId`, `text` (<= 280–4000 chars configurable), `createdAt`, `inReplyToId?`, `quoteOfId?`, `conversationId` (root of thread), `mediaRefs[]` (array of `mediaId`), `entities{hashtags[], mentions[], urls[]}`, `visibility` (public/protected), `language`, `geo?` (point + `placeId`), `flags` (deleted, edited), `editGroupId?` (for edit history), **counters** `{likeCount, retweetCount, replyCount, viewCount}`.
- Typical queries: author's timeline (authorId recent), by `tweetId`, conversation fetch by `conversationId`.
- Consistency: write-ack on quorum; **read-your-writes** via session stickiness; counters eventually consistent with periodic reconcile

eventually consistent with periodic reconcile.

7.3 Favorites:

This table maps tweets with users for the favorite tweets functionality in our application.

```
Favorites
{ id: uuid,
 UserID: uuid,
TweetID: uuid,
CreatedAt: timestamp
```

7.4 Followers:

This table maps the followers and followess (one who is followed) as users can folloe each other. The relation will be N:M relationship.

```
Followers
{
id: uuid,
followerID: uuid,
 followeeID: uuid,
}
```

7.5 Feeds:

This table stores feed properties with the corresponding userID.

```
Feeds
{
 id: uuid,
  userID,
 UpdatedAt: Timestamp
}
```

7.6 Feeds_Tweets:

This table maps tweets and feed. There relation will be (N:M relationship).

```
feeds_tweets{
  id: uuid,
 tweetID: uuid,
 feedID: uuid
}
```

7.7 What Kind of Database is used in Twitter?

- While our data model quite relational, we don't necessarily need to store everything in a single database, as this can limit our scalability and quickly become a bottleneck.
- We will split the data between different services each having ownership over a particular table. The we can use a realtional database such as PostgreSQL or a distributed NoSQL database such as Apache Cassandra PostgreSQL for our usecase.

7.8 Core Entities (Logical Model – Production)

- User(userId, handle, name, email, bio, createdAt, settings, statusFlags)
- Tweet(tweetId, authorId, text, createdAt, inReplyToId?, quoteOfId?, mediaRefs[], entities{hashtags[], mentions[]}, visibility, language, geo?)
- FollowEdge(followerId, followeeId, createdAt, state)
- Like(userId, tweetId, createdAt)
- Retweet(userId, srcTweetId, createdAt, retweetId)
- TimelineEntry(userId, tweetId, rankScore, insertedAt, source) – materialized home timeline
- Notification(userId, type, actorId, subjectId, createdAt, state)
- TrendCounter(bucket, hashtag/topic, region, count)
- AbuseSignals(subjectType, subjectId, signalType, score, bucket)

7.9 Indexing & Sharding Keys (Practical)

- Tweets partitioned by authorId hash (write locality); secondary indexes: by tweetId, by createdAt.
- Social graph edges partitioned by followerId (for read: “who I follow”) and by followeeId (for fanout).
- TimelineEntry partitioned by userId.
- Search is separate inverted index (text + entities).

7.10 Storage Choices (Operational)

- OLTP: wide-column (Cassandra-like) for tweets/timelines/edges or a Twitter-style multi-tenant store;
- Relational store for identity & strong-consistency domains (handles, email, auth).
- Object storage for media; CDN for egress; thumbnails/variants stored & cached.

8. API Design for Twitter System Design

A basic API design for our services:

8.1 Post a Tweets:

8.1 Post a Tweets:

This API will allow the user to post a tweet on the platform.

```
{
userID: UUID,
   content: string,
  mediaURL?: string
}
```

- **User ID (`UUID`):** ID of the user.
- **Content (`string`):** Contents of the tweet.
- **Media URL (`string`)** : URL of the attached media *(optional)* .
- **Result (`boolean`)** : Represents whether the operation was successful or not.

8.2 Follow or unfollow a user

This API will allow the user to follow or unfollow another user.

Follow Unfollow

```
{
  followerID: UUID,
  followeeID: UUID
}
```

Parameters

- **Follower ID (UUID):** ID of the current user.
- **Followee ID (UUID):** ID of the user we want to follow or unfollow.
- **Media URL (string):** URL of the attached media (optional).
- **Result (boolean)** : Represents whether the operation was successful or not.

8.3 Get NewsFeed

This API will return all the tweets to be shown within a given newsfeed.

```
{
  userID: UUID
}
```

Parameters

- **User ID (UUID):** ID of the user.
- **Tweets (Tweet[]):** All the tweets to be shown within a given newsfeed.

9. Microservices Used for Twitter System Design

9.1 Data Partitioning

To scale out our databases we will need to partition our data. Horizontal partitioning (aka Sharding) can be a good first step. We can use partitions schemes such as:

- Hash-Based Partitioning
- List-Based Partitioning
- Range Based Partitioning
- Composite Partitioning

The above approaches can still cause uneven data and load distribution, we can solve this using Consistent hashing.

9.2 Mutual friends

- For mutual friends, we can build a social graph for every user. Each node in the graph will represent a user and a directional edge will represent followers and followees.
- After that, we can traverse the followers of a user to find and suggest a mutual friend. This would require a graph database such as Neo4j and ArangoDB.
- This is a pretty simple algorithm, to improve our suggestion accuracy, we will need to incorporate a recommendation model which uses machine learning as part of our algorithm.

9.3 Metrics and Analytics

- Recording analytics and metrics is one of our extended requirements.
- As we will be using Apache Kafka to publish all sorts of events, we can process these events and run analytics on the data using Apache Spark which is an open-source unified analytics engine for large-scale data processing.

9.4 Caching

- In a social media application, we have to be careful about using cache as our users expect the latest data. So, to prevent usage spikes from our resources we can cache the top 20% of the tweets.
- To further improve efficiency we can add pagination to our system APIs. This decision will be helpful for users with limited network bandwidth as they won't have to retrieve old messages unless requested.

9.5 Media access and storage

- As we know, most of our storage space will be used for storing media files such as images, videos, or other files. Our media service will be handling both access and storage of the user media files.
- But where can we store files at scale? Well, object storage is what we're looking for. Object stores break data files up into pieces called objects.
- It then stores those objects in a single repository, which can be spread out across multiple networked systems. We can also use distributed file storage such as HDFS or GlusterFS.

9.6 Content Delivery Network (CDN)

- Content Delivery Network (CDN) increases content availability and redundancy while reducing bandwidth costs.
- Generally, static files such as images, and videos are served from CDN. We can use services like Amazon CloudFront or Cloudflare CDN for this use case.

10. Scalability for Twitter System Design

Let us identify and resolve Scalability such as single points of failure in our design:

- "What if one of our services crashes?"
- "How will we distribute our traffic between our components?"
- "How can we reduce the load on our database?"
- "How to improve the availability of our cache?"
- "How can we make our notification system more robust?"
- "How can we reduce media storage costs"?

To make our system more resilient we can do the following:

- Running multiple instances of each of our services.
- Introducing load balancers between clients, servers, databases, and cache servers.
- Using multiple read replicas for our databases.
- Multiple instances and replicas for our distributed cache.
- Exactly once delivery and message ordering is challenging in a distributed system, we can use a dedicated message broker such as Apache Kafka or NATS to make our notification system more robust.
- We can add media processing and compression capabilities to the media service to compress large files which will save a lot of storage space and reduce cost.

11. Caching Strategy

- **Frontline**: CDN for media & static assets; HTTP cache headers.
- **Data**: Redis/Memcache for tweet bodies, user profiles, timeline pages; use write-through for counters; read-through with TTL + jitter; negative-cache 404s briefly.

- **Key invalidation**: on tweet edit/delete, on follow changes, on engagement thresholds.

12. Partitioning & Hot Keys

- Use per-author queues to smooth spikes; backpressure and QoS.

13. Consistency Model

- **Strong**: identity (handles/emails), payments.
- **Eventual**: home timeline materialization, counts, trends.
- **Read-your-writes**: route user to home region; session stickiness or client-side merge to show newly sent tweet immediately.

14. Reliability, SRE & Observability

- **SLIs/SLOs**: availability, latency (p50/p95/p99), error rate, freshness (indexing delay), delivery success.
- **Dashboards** per surface; RED/USE metrics.
- **Tracing** with end-to-end IDs; sampling for hot paths.
- Circuit breakers and bulkheads per service.
- **Chaos testing**: kill brokers, drop partitions; verify graceful degradation (e.g., fallback to latest unranked timeline).
- **Disaster Recovery**: multi-region; RPO minutes for timelines (rebuildable), RPO≈0 for tweet log; RTO < 1 hour via runbooks.

15. Security, Privacy & Abuse

- OAuth2/OIDC, short-lived tokens. TLS-only. At-rest encryption (KMS).
- **Rate limiting** per IP/user/app; dynamic shields during events.
- **Abuse/Spam**: ML signals (account age, device fingerprints, graph patterns), risk scoring on write; shadow-ban/quarantine queues; user reporting workflows.
- **Privacy**: blocks/mutes filtering in all reads; Right-to-Erasure: tombstones + GC in OLTP, deindex in search, purge from caches/CDN; data retention policy by region (GDPR/CCPA).
- **Content safety**: media hashing (perceptual), NSFW classifiers; age gates.

16. Cost Controls

- Push more through CDN (thumbs/variants, far-future TTL).
- Storage tiers & compaction; compress text; column families for hot/cold fields.
- Compute autoscaling on QPS/backlog; spot/preemptible for batch.
- Defer heavy recompute (e.g., rank refresh) for low-value cohorts.

17. Failure Modes & Mitigations (Interview-Gold)

- **Hot celebrity tweet**: push for normal users, pull for celebrity; cap fanout concurrency, enqueue per-follower buckets; protect caches from thundering herds with request collapse.
- **Search indexing lag**: serve latest via realtime tier; degrade to “latest only” if top tier fails.
- **Cache outage**: hit stores with read fences + partial pagination; reduce page size; disable ranked sort.
- **Kafka partition loss**: dual-write to mirror; consumer offsets backed by transactional store.

18. E2E Sequence (Tweet → Follower Home)

```
Client → API → TweetSvc: validate, allocate id
TweetSvc → Store: write quorum OK
TweetSvc → Bus: publish TweetCreated
FanoutWorker (consumes):
  - fetch followers
  - for non-celebs: append TimelineEntry(userId,tweetId,rankSeed)
  - for celebs: mark home-shard as needs-merge
SearchIndexer: tokenize, add to realtime index
NotificationSvc: build mention/author notifications
Client GET /timeline/home → TimelineSvc:
  - read cached page or build page
  - join tweets, run rank, filter blocks/mutes
  - return page + cursor
```

19. Schema Sketches (illustrative)

```
Tweet{ tweetId PK, authorId, text, createdAt, replyToId?, quoteOfId?,
       media[], lang, visibility, entities{hashtags[],mentions[]} }

FollowEdge{ followerId, followeeId, createdAt, state,  PRIMARY KEY(followerId, followeeId)
}

TimelineEntry{ userId, tweetId, insertedAt, rankScore, source,
               PRIMARY KEY(userId, insertedAt DESC, tweetId) }

Like{ userId, tweetId, createdAt, PRIMARY KEY(userId, tweetId) }

Retweet{ userId, srcTweetId, retweetId, createdAt,
         PRIMARY KEY(userId, srcTweetId) }

Notification{ userId, notifId, type, actorId, subjectId, createdAt, state }
```

20. Search Relevance Features (starter)

- Text: BM25 features, exact hashtag/mention boosts, URL domain quality.
- Social: author authority, user-author affinity, engagement freshness.
- Safety: language/NSFW, policy scores.
- Learning-to-rank: LambdaMART/XGBoost; online features from streams.